

Lesson 6 — Intro to UML & Subsystems



  RamRodz Robotics · Team 6366 · Programming Worksheet

NAME _____

DATE _____

  RAM RODZ ROBOTICS | TEAM #6366 | HILLCREST & WOODMONT HIGH SCHOOLS · SIMPSONVILLE, SC

 Focus Check-In:

 Struggling

 Okay

 Ready

TODAY'S BIG QUESTION

Your robot has six motors and two controllers — how do engineers keep all that code organized without losing their minds?

Parking Lot — Random Thoughts

1 Key Words — Check Off When Explained



UML

Unified Modeling Language — a standard diagram system for showing how code is structured, like a blueprint before you build

Private (-)

Only the class itself can read or change this — shown as a minus sign in UML. Used for fields to protect the data

Class Diagram

A UML diagram with three sections: the class name on top, its fields in the middle, and its methods on the bottom

Public (+)

Any code outside the class can use this — shown as a plus sign in UML. Methods are usually public so others can call them

Access Modifier

A keyword that controls who can use a field or method — either `public` or `private` in Java

Subsystem

A class that controls one part of the robot — such as the intake, shooter, climber, or drivetrain

2 Reading a UML Class Diagram



3 Step by Step — Build IntakeSubsystem



4 Your Turn — Build ShooterSubsystem





END-GAME SITUATION

With 20 seconds left your driver presses a button to deploy the climber. The arm must extend to a target position, then retract safely when done. Your coach needs a way to check from outside the class whether the climber is still deployed.

- The arm is driven by a **PWMSparkMax** on PWM port 6
- Track the arm's position (`double`) and whether it's out (`boolean`)
- `deploy(position)` takes a `double` parameter — extend to that position
- `retract()` brings the arm back to 0.0 and marks it not deployed
- `isDeployed()` returns the current deployed state

ClimberSubsystem

```
- armMotor      : PWMSparkMax
- armPosition   : double
- deployed      : boolean

+ ClimberSubsystem()
+ deploy(position : double) : void
+ retract()          : void
+ isDeployed()       : boolean
```

 **New this time:** `deploy(position : double)` has a **parameter**. In Java that becomes `public void deploy(double position)`. Use the parameter like any variable inside the method body — store it in `armPosition` and pass it to `armMotor.set()`.

 **Fields — write all three private field declarations**

 **Constructor** — initialize all three fields (motor on PWM port 6)

 **deploy() Method** — takes a double parameter named position

 **retract() Method** — bring the arm home

 **isDeployed() Method** — returns a boolean

☐ I built ClimberSubsystem from its UML — including a method with a parameter.

✓ End of Lesson — Self Check

- ☐ I can read a UML class diagram and identify the name, fields, and methods.
- ☐ I know that `-` in UML means `private` and `+` means `public` in Java.
- ☐ I can translate a UML diagram into a Java class with fields, a constructor, and methods.
- ☐ I understand why fields are usually `private` and methods are usually `public`.
- ☐ I built ClimberSubsystem on my own from a UML diagram — including a method with a parameter.